PRUEBA

Documentación Técnica

Generado: 16/10/2026 03:40 UTC

AutoDocs AI

Tabla de Contenido

Tabla de Contenido 2

Documentación del Proyecto 9

Resumen Ejecutivo . . . 9

Stack Tecnológico . . . 9

Estadísticas del Proyecto . . . 9

2. Documentación de Arquitectura y Diseño . . . 10

2.1 Estructura de Directorios . . . 10

2.2 Distribución por Capas . . . 10

2.3 Distribución de Lenguajes . . . 10

2.4 Diagrama de Infraestructura . . . 11

2.5 Diagrama de Casos de Uso . . . 12

2.6 Diagrama de Clases . . . 12

2.7 Patrones Detectados . . . 13

2.8 Insights de Arquitectura (IA) . . . 13

3. Documentación Técnica de Código . . . 14

3.1 Guía de Configuración del Entorno de Desarrollo . . . 14

3.2 Documentación de API . . . 14

3.3 Estado de Comentarios y Docstrings . . . 18

3.4 Planes de Prueba (Testing) . . . 20

4. Documentación de Clases . . . 21

`User` . . . 21

1. Propósito de la clase (en palabras muy simples) . . . 21

2. Qué representa esta clase en el sistema (una analogía del día a día) 22

3. Qué puede hacer esta clase (funcionalidades principales explicadas de forma sencilla) . 22

5. Documentación de Funciones 22

Módulo: `controllers/userController.py` 22

Módulo: `models/User.py` 24

Módulo: `routes/user.py` 25

6. Modelos de Datos 30

Esquema General Inferido 31

7. Guía de Despliegue y Mantenimiento 31

7.1 Requisitos del Sistema 31

7.2 Instalación Local 31

7.3 Despliegue con Docker 31

7.4 Variables de Entorno Recomendadas 32

7.5 Checklist de Despliegue 32

7.6 Registro de Cambios (Changelog) 33

8. Reporte de Calidad del Código 33

8.1 Análisis de Métricas 33

8.2 Funciones con Alta Complejidad Ciclomática 34

8.3 Recomendaciones 34

8.4 Errores de Análisis 34

9. Análisis de Datos del Proyecto 34

9.1 Tipos y Fuentes de Datos Detectados 34

9.2 Esquema de Data Warehouse Inferido 34

9.3 Reporte de Calidad de Datos 35

9.4 Minería de Datos Aplicada al Código 35

9.5 Pipeline ETL Utilizado 36

prueba

Documentación Técnica · AutoDocs AI · 16/10/2026

8	10	1	4
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 82/100

Auditoría & Especificación Técnica de Software

<h1 class="cover-title" style="string-set: project-name content();">Proyecto</h1>

Documento Maestro de Arquitectura, Endpoints, Modelos de Dominio y Métricas de Calidad

<table class="meta-table">

<tr><td>Proyecto:</td><td>Proyecto</td></tr>

<tr><td>Lenguaje Dominante:</td><td>Python</td></tr>

<tr><td>Score de Calidad:</td><td>82 / 100 (Excelente)</td></tr>

<tr><td>Total Archivos Analizados:</td><td>8</td></tr>

<tr><td>Fecha de Generación:</td><td>2026-10-16 03:39 UTC</td></tr>

<tr><td>Motor de Generación:</td><td>AutoDocs AI Enterprise (Groq IA + PlantUML/Kroki)</td></tr>

</table>

8Archivos

10Funciones

1Clases

4Endpoints

Documento técnico confidencial · Generado automáticamente por AutoDocs AI · Distribución interna

<style>a.toc-page{text-decoration:none;}
a.toc-page::after{content:target-counter(attr(href),page);}</style>

<h1>Tabla de Contenido</h1>

Índice navegable de capítulos, secciones técnicas y diagramas arquitectónicos.

Documentación del Proyecto

Resumen Ejecutivo

Stack Tecnológico

Estadísticas del Proyecto

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

2.2 Distribución por Capas

2.3 Distribución de Lenguajes

2.4 Diagrama de Infraestructura

2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases

2.7 Patrones Detectados

2.8 Insights de Arquitectura (IA)

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

3.2 Documentación de API

3.3 Estado de Comentarios y Docstrings

3.4 Planes de Prueba (Testing)

4. Documentación de Clases

User

1. Propósito de la clase (en palabras muy simples)

2. Qué representa esta clase en el sistema (una analogía del día a día)

3. Qué puede hacer esta clase (funcionalidades principales explicadas de forma sencilla)

5. Documentación de Funciones

Módulo: controllers/userController.py

Módulo: models/User.py

Módulo: routes/user.py

6. Modelos de Datos

Esquema General Inferido

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

7.2 Instalación Local

7.3 Despliegue con Docker

7.4 Variables de Entorno Recomendadas

7.5 Checklist de Despliegue

7.6 Registro de Cambios (Changelog)

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

8.2 Funciones con Alta Complejidad Ciclomática

8.3 Recomendaciones

8.4 Errores de Análisis

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

9.2 Esquema de Data Warehouse Inferido

9.3 Reporte de Calidad de Datos

9.4 Minería de Datos Aplicada al Código

9.5 Pipeline ETL Utilizado

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-10-16 03:39 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con
8 archivos fuente, 10 funciones, 1 clases y
4 endpoints de API documentados. El score de calidad del código es
82/100.

Stack Tecnológico

Lenguaje	Cantidad
Python	8 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	8
Funciones	10
Clases	1
Endpoints API	4

Score de calidad	82/100
Funciones documentadas	4 / 10
Clases documentadas	0 / 1

---

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ controllers/
■■■ models/
■■■ routes/
```

2.2 Distribución por Capas

• `(raíz)/` — 6 archivos [.env(1), .md(1), .py(2), .gitignore(1), .txt(1)]

• `controllers/` — 2 archivos [.py(2)]

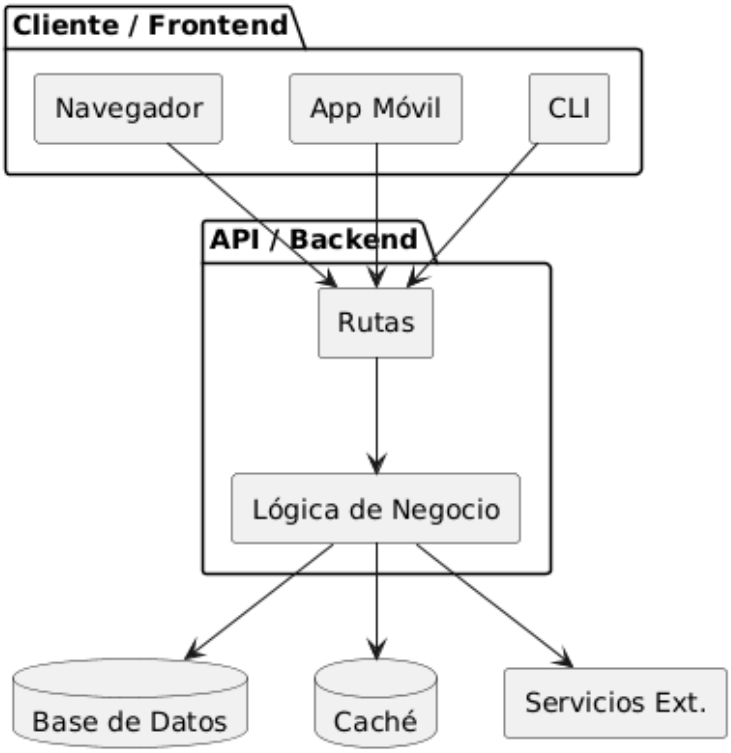
• `models/` — 2 archivos [.py(2)]

• `routes/` — 2 archivos [.py(2)]

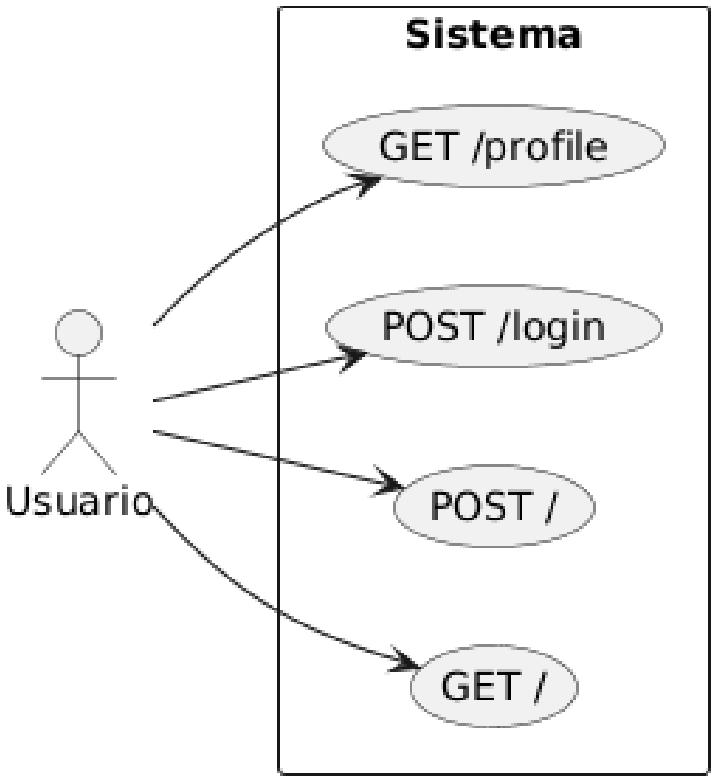
2.3 Distribución de Lenguajes

• Python: 8 archivos (100%)

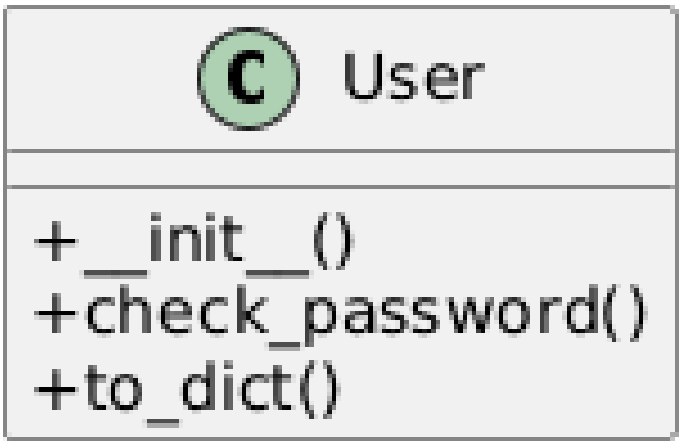
2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



2.7 Patrones Detectados

• **API REST**: 4 endpoints detectados (Flask)

• **Programación Orientada a Objetos**: constructores detectados

2.8 Insights de Arquitectura (IA)

1. Patrones arquitectónicos probables

• Con solo 1 clase y 4 endpoints, lo más probable es un **monolito ligero** o una **API estilo "micro■framework"** (Flask/FastAPI) sin separación estricta de capas.

• La ausencia de vistas o controladores explícitos sugiere que no se está aplicando MVC; la lógica probablemente está mezclada en los handlers de los endpoints.

2. Buenas prácticas detectadas

• Uso exclusivo de Python indica consistencia de lenguaje.

• Número reducido de funciones (10) y clases (1) facilita la lectura y el testing unitario.

• La presencia de endpoints muestra que la aplicación está orientada a servicios HTTP, lo cual es apropiado para APIs.

3. Áreas de mejora sugeridas

• **Separación de responsabilidades**: crear capas de *service* y *repository* para aislar la lógica de negocio y el acceso a datos.

• **Modularidad**: dividir la única clase en componentes más pequeños (p. ej., DTOs, validadores).

• **Documentación**: añadir OpenAPI/Swagger para describir los 4 endpoints.

• **Manejo de errores**: centralizar excepciones y respuestas de error.

4. Complejidad del proyecto

• Baja complejidad estructural (solo 10 funciones, 1 clase).

• La complejidad lógica interna no se puede medir, pero el tamaño sugiere un riesgo bajo de deuda técnica inicial.

5. Recomendaciones de escalabilidad

• **Adoptar una arquitectura en capas** (API → Service → Repository) para facilitar la

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **4**

Módulo: `routes/user.py`

GET `GET /`

• **Framework**: Flask

• **Archivo**: `routes/user.py`

1. Propósito del endpoint (¿para qué sirve?)

Este recurso es la “puerta de entrada” que permite a cualquier persona o aplicación preguntar al sistema quiénes son los usuarios registrados. En otras palabras, sirve para obtener una lista de los usuarios que existen en la aplicación.

2. Qué hace este endpoint (funcionalidad)

Cuando se le hace una petición, el programa busca en su base de datos (o en la lista interna) todos los usuarios que están guardados y los reúne en un formato que cualquiera puede leer. No cambia nada, solo **lee** la información y la devuelve.

3. Qué información necesita para funcionar

Este endpoint no necesita que le envíes nada especial. Sólo tiene que recibir la solicitud (un simple “dame la lista”). No se le pasan parámetros, contraseñas ni datos adicionales.

4. Qué resultado devuelve

Devuelve una respuesta que contiene la lista de usuarios. Cada usuario suele aparecer con datos básicos como su nombre, su correo electrónico y su identificador único. La respuesta está estructurada de forma que tanto humanos como otras aplicaciones puedan entenderla (por ejemplo, en formato JSON).

5. Para qué se usa en la vida real (ejemplos prácticos)

| Situ

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

POST `POST /`

• **Framework**: Flask

• **Archivo**: `routes/user.py`

1. Propósito del endpoint (¿para qué sirve?)

Este punto de acceso — una URL a la que se envía información — permite **crear o registrar un nuevo usuario** en la aplicación. Piensa en él como el formulario que llenas cuando te apuntas a un sitio web: al enviarlo, el sistema guarda tus datos

para que puedas usar la plataforma.

2. Qué hace este endpoint (funcionalidad)

Cuando recibe una petición **POST** (es decir, “aquí tienes datos que quiero que guardes”), el código que está en el archivo `routes/user.py` toma la información que le envías, la verifica (por ejemplo, que el nombre no esté vacío y que la contraseña tenga la longitud mínima) y, si todo está correcto, la almacena en la base de datos del proyecto. En resumen: **toma los datos del nuevo usuario y los guarda**.

3. Qué información necesita para funcionar (entrada)

Para que el endpoint pueda crear el usuario, debes enviarle un pequeño paquete de datos en formato JSON (un texto estructurado). Los campos más habituales son:

|

Parámetros esperados:

• `body` (JSON, requerido): Cuerpo de la solicitud

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

POST `POST /login`

• **Framework**: Flask

• **Archivo**: `routes/user.py`

1. Propósito del endpoint

Este punto de la aplicación sirve para que un usuario pueda **entrar** a su cuenta. Es como la puerta de entrada de una casa: cuando alguien escribe su nombre de usuario y su contraseña, el sistema verifica que esos datos sean correctos y, si lo son, le permite pasar al interior.

2. Qué hace este endpoint

• Recibe los datos que el usuario escribe (normalmente *usuario* y *contraseña*).

• Comprueba que esos datos coincidan con los que están guardados en la base de datos del proyecto.

• Si todo está bien, crea una “llave” temporal (un token) que identifica al usuario durante su sesión y la envía de vuelta.

• Si algo falla (por ejemplo, la contraseña está equivocada), le dice al usuario que no pudo entrar.

3. Qué información necesita para funcionar

Para poder trabajar, el endpoint necesita que le envíes un **mensaje** (un “payload”) con dos cosas:

Campo	Qué significa
`username` (o `email`)	El nombre o correo con el que el usuario se registró.
`password`	La clave secreta que el usuario eligió al crear su

Parámetros esperados:

• **`body`** (JSON, requerido): Cuerpo de la solicitud

• **`Authorization`** (Header, requerido): Bearer JWT token

Códigos de respuesta:

• **`200`** — Operación exitosa

• **`400`** — Solicitud incorrecta

• **`401`** — No autorizado

• **`404`** — Recurso no encontrado

• **`500`** — Error interno del servidor

GET **`GET /profile`**

• **Framework**: Flask

• **Archivo**: **`routes/user.py`**

Parámetros esperados:

• **`Authorization`** (Header, requerido): Bearer JWT token

Códigos de respuesta:

• **`200`** — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 10

• OK **Documentadas:** 4 (40%)

• Pendiente **Sin documentar:** 6 (60%)

Visualización de documentación:

[■■■■■■■■■■■■■■■■■■■■] 40% documentado

Ejemplo de docstrings detectados:

```
def index():
    """
    Obtener todos los usuarios
---
tags:
  - Usuarios
responses:
  200:
    description: Lista de usuarios
    """
    ...
```

```
def user_store():
    """
    Crear un usuarios
---
tags:
  - Usuarios
parameters:
  - in: body
    name: user
```

```
    schema:
      type: object
      required:
        - name
        - email
        - password
      properties:
        name:
          type: string
          example: "Juan Pérez"
        email:
          type: string
          example: "juan@example.com"
        password:
          type: string
          example: "123456"
responses:
  201:
    description: Usuario creado
    """
    ...
```

```
def login():
    """
    Iniciar sesión
---
tags:
  - Autenticación
parameters:
  - in: body
    name: credentials
    schema:
      type: object
      required:
        - email
        - password
      properties:
        email:
          type: string
          example: "juan@example.com"
        password:
          type: string
          example: "123456"
responses:
  200:
    description: Login exitoso
    """
```

```
    ...
```

Funciones que requieren documentación:

- `get_all_users()` en `controllers/userController.py`
- `create_user(name, email, password)` en `controllers/userController.py`
- `login_user(email, password)` en `controllers/userController.py`
- `__init__(self, name, email, password)` en `models/User.py`
- `check_password(self, password)` en `models/User.py`
- ... y 1 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

| Escenario | Tipo | Prioridad |

| --- | --- | --- |

| GET / — respuesta 200 | Integración | Alta |

| GET / — sin auth (401) | Seguridad | Alta |

| POST / — respuesta 200 | Integración | Alta |

| POST / — sin auth (401) | Seguridad | Alta |

| POST /login — respuesta 200 | Integración | Alta |

| POST /login — sin auth (401) | Seguridad | Alta |

| GET /profile — respuesta 200 | Integración | Alta |

| GET /profile — sin auth (401) | Seguridad | Alta |

---

4. Documentación de Clases

Total: **1 clases**

`User`

• **Archivo:** `models/User.py`

• **Línea:** 4

• **Hereda de:** db.Model

• **Métodos:** 3

• **Documentada:** No

Clase : User

Archivo : models/User.py

---

1. Propósito de la clase (en palabras muy simples)

Imagina que la aplicación necesita “guardar” a las personas que se registran para usarla.

La clase **User** es la “plantilla” que usamos para crear esos “registros de usuario”. Cada vez que alguien se crea una cuenta, se crea un objeto *User* que contiene la información esencial de esa persona.

---

2. Qué representa esta clase en el sistema (una analogía del día a día)

Piensa en un **carné de socio** de un club.

• En el carné aparecen datos como el nombre, el correo y una clave secreta (contraseña).

• El carné también permite comprobar que quien lo muestra es realmente el socio (verificando la contraseña).

La clase **User** es el "carné digital" que la aplicación entrega a cada usuario.

---

3. Qué puede hacer esta clase (funcionalidades principales explicadas de forma sencilla)

| Acción | Qué hace realmente la clase | Explicación sencilla |

|--------|----------------------------|----------------------|

| **Crear un nuevo

Métodos:

• `__init__()`

• `check_password()`

• `to_dict()`

---

5. Documentación de Funciones

Total: **10 funciones**

Módulo: `controllers/userController.py`

`get_all_users`

• Parámetros: `–`

• Async: —

• Complejidad: Baja 4

• Documentada: No

> **La función `get_all_users` actúa como un catálogo maestro que recupera la información de todos los registros almacenados en el sistema. No requiere datos de entrada específicos, ya que su diseño busca obtener la totalidad de la base de datos de usuarios automáticamente. Como resultado, entrega una lista completa con los perfiles de cada usuario registrado, incluyendo sus detalles de identificación.**

En la vida real, esta herramienta es vital para tareas administrativas, como generar reportes de membresía, auditar cuentas activas o facilitar funciones de búsqueda global en paneles de control. Su importancia radica en proporcionar una visión centralizada y actualizada de la comunidad o clientes del sistema, permitiendo a los administradores tomar decisiones informadas sin necesidad de consultar fichas individuales. Es, esencialmente, la llave maestra para gestionar la totalidad de la base de usuarios.

`create_user`

• Parámetros: `name, email, password`

• Async: —

• Complejidad: Baja 3

• Documentada: No

> **La función `create_user` actúa como el "registro inicial" de una persona en el sistema. Su propósito es registrar nuevos usuarios de forma segura y organizada.**

Para funcionar, necesita tres datos básicos: el nombre de la persona, su correo electrónico y una contraseña. Con esta información, el sistema crea un perfil único asociado a ese usuario.

El resultado es la creación de una cuenta activa que permite a la persona iniciar sesión y acceder a servicios personalizados. En la vida real, esto es lo que ocurre cuando te registras en una tienda en línea, un banco digital o una red social.

Su importancia es crítica, ya que es la puerta de entrada a toda la plataforma. Sin esta función, nadie podría tener identidad dentro del sistema, ni realizar compras, ver contenido ni gestionar su perfil. Es la base fundamental para cualquier interacción posterior.

`login_user`

• Parámetros: `email, password`

• Async: —

• Complejidad: Baja 3

• Documentada: No

> **La función `login_user` actúa como la cerradura de seguridad de la aplicación, permitiendo que los usuarios accedan a su cuenta personal. Para operar, requiere dos datos específicos: el correo electrónico y la contraseña que el usuario proporcionó al registrarse. El sistema verifica esta información contra los registros almacenados. Si los datos son correctos, genera una confirmación de acceso que habilita la sesión del usuario; si son incorrectos, devuelve un mensaje de error. En la vida real, esto es lo que ocurre cuando inicias sesión en redes sociales, bancos en línea o correos electrónicos. Su importancia es crítica, ya que es el primer punto de control que garantiza que solo las personas autorizadas puedan entrar al sistema, protegiendo la privacidad y la seguridad de la información de los usuarios.**

Módulo: `models/User.py`

`__init__`

• Parámetros: `self, name, email, password`

• Async: —

• Complejidad: Baja 1

• Documentada: No

> **Esta función actúa como la plantilla inicial para crear un nuevo perfil de usuario en la aplicación. Su propósito es preparar la "ficha" de identidad antes de que el sistema guarde la información.**

Para funcionar, requiere tres datos esenciales: el nombre de la persona, su correo electrónico y una contraseña secreta. Estos elementos definen quién es el usuario y cómo ingresará al sistema.

El resultado es la creación de una estructura de datos organizada en memoria, lista para ser almacenada en la base de datos. No guarda la información por sí sola, sino que define su forma y contenido.

En la vida real, se usa cuando alguien se registra en una red social, banco o tienda online. Cada vez que haces "crear cuenta", esta función se ejecuta para organizar tus datos.

Su importancia es crítica: sin ella, el sistema no podría identificar ni diferenciar a los usuarios. Es la base de la seguridad y la personalización de la experiencia,

asegurando que cada persona tenga un espacio único y protegido dentro de la plataforma.

`check_password`

• Parámetros: `self, password`

• Async: —

• Complejidad: Baja 1

• Documentada: No

> **check_password es la herramienta de seguridad que verifica si la contraseña introducida por un usuario es correcta antes de permitir el acceso a su cuenta.**

Para funcionar, necesita dos elementos: la referencia a la cuenta del usuario (para conocer su contraseña guardada de forma segura) y la contraseña que la persona acaba de escribir.

El resultado es una respuesta simple: confirma si la contraseña coincide o no. Si coincide, el sistema permite continuar; si no, bloquea el intento.

En la vida real, se usa cada vez que inicias sesión en un correo, banco o red social. Es el "portero" digital que asegura que solo tú puedes entrar.

Su importancia es crítica: sin ella, cualquiera podría acceder a tus datos personales sin tu permiso, comprometiendo la privacidad y la seguridad de toda la plataforma.

`to_dict`

• Parámetros: `self`

• Async: —

• Complejidad: Baja 1

• Documentada: No

Módulo: `routes/user.py`

`index`

• Parámetros: `–`

• Async: —

• Complejidad: Baja 1

• Documentada: Sí

Obtener todos los usuarios

tags:

- Usuarios

responses:

200:

description: Lista de usuarios

`user_store`

• Parámetros: **–**

• Async: —

• Complejidad: Baja 1

• Documentada: Sí

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name

- email

- password

properties:

name:

type: string

example: "Juan Pérez"

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

201:

description: Usuario creado

`login`

• Parámetros: –

• Async: —

• Complejidad: Baja 1

• Documentada: Sí

Iniciar sesión

tags:

- Autenticación

parameters:

- in: body

name: credentials

schema:

type: object

required:

- email

- password

properties:

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

200:

description: Login exitoso

`profile`

• Parámetros: **—**

• Async: —

• Complejidad: Baja 2

• Documentada: Sí

Obtener perfil del usuario autenticado

tags:

- Usuarios

responses:

200:

description: Datos del usuario

`index()`

Ubicación: `routes/user.py`

Obtener todos los usuarios

tags:

- Usuarios

responses:

200:

description: Lista de usuarios

`user_store()`

Ubicación: `routes/user.py`

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name

- email

- password

properties:

name:

type: string

example: "Juan Pérez"

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

201:

description: Usuario creado

`login()`

Ubicación: `routes/user.py`

Iniciar sesión

---

tags:

- Autenticación

parameters:

- in: body

name: credentials

schema:

type: object

required:

- email

- password

properties:

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

200:

description: Login exitoso

`profile()`

Ubicación: `routes/user.py`

Obtener perfil del usuario autenticado

---

tags:

- Usuarios

responses:

200:

description: Datos del usuario

---

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

• **All Users**

• **User**

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

• **Lenguaje principal**: Python

• **Archivos detectados**: 8

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
```

```
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [ ] Variables de entorno configuradas
- [ ] Base de datos migrada
- [ ] Tests ejecutados y pasando
- [ ] Logs configurados
- [ ] HTTPS habilitado en producción
- [ ] Backups programados

• [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-16

• Versión inicial analizada por AutoDocs AI

• Documentación técnica generada automáticamente

• Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 82/100 Excelente

Distribución de Documentación:

• Funciones documentadas: 4/10 (40%)

• Clases documentadas: 0/1

[████████████████████] 40% documentado

Análisis de Complejidad:

• Complejidad promedio: 1.8

• Complejidad máxima: 4

• Funciones con alta complejidad (>10): 0

• Tendencia de complejidad: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10.

8.3 Recomendaciones

• Agregar docstrings a las 6 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados.

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• **Funciones sin documentar:** 6

- `get_all_users` en `controllers/userController.py`
- `create_user` en `controllers/userController.py`
- `login_user` en `controllers/userController.py`
- `__init__` en `models/User.py`
- `check_password` en `models/User.py`
- ... y 1 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
Alta	> 5	0	Requieren refactorización urgente
Media	4–5	1	Candidatas a simplificación
Baja	≤ 3	9	Complejidad aceptable

No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

6 de 10 funciones carecen de docstring (60%).

Función	Archivo	Línea	Complejidad
`get_all_users`	`controllers/userController.py`	6	4
`create_user`	`controllers/userController.py`	13	3
`login_user`	`controllers/userController.py`	23	3
`__init__`	`models/User.py`	10	1
`check_password`	`models/User.py`	15	1
`to_dict`	`models/User.py`	18	1

Sugerencia: Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

No se detectaron funciones con nombres similares en archivos distintos.

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

vo	Funciones	Complejidad Promedio	Documentadas	Estado
/user.py`	4	1.2	100%	OK<
llers/userController.py`	3	3.3	0%	OK<
s/User.py`	3	1.0	0%	OK<

E) Patrones Frecuentes Adicionales (Análisis IA)

• **init**: 2 ocurrencias

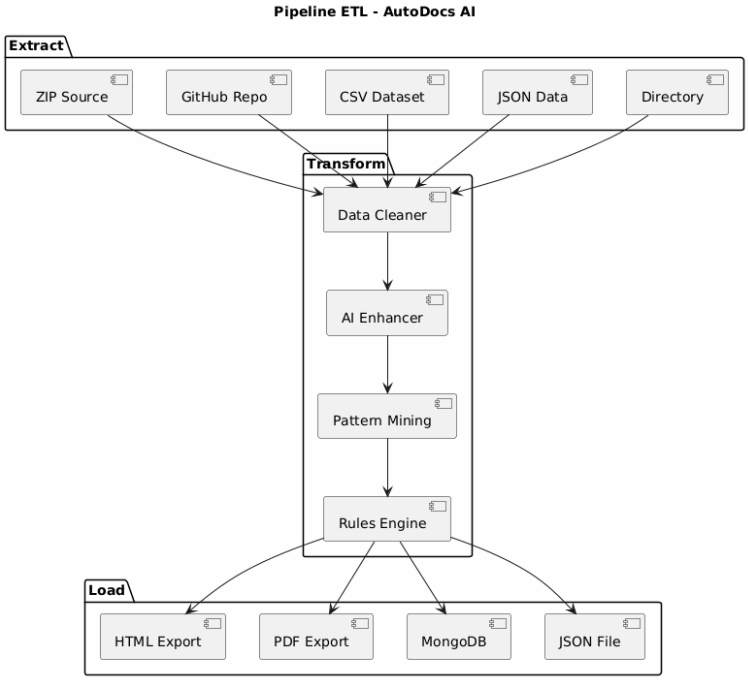
- Ejemplos: ['__init__', 'init']

• **user**: 5 ocurrencias

- Ejemplos: ['create_user', 'login_user', 'get_all_users', 'User', 'user_store']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

• **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales

• **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones

• **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-16 03:39 UTC
